

QA Testing Guide

AI Career Growth & Talent Intelligence Platform — Backend (Phase 1, Phase 2 & Phase 3)

Backend handoff for the frontend & QA teams — API contracts (Section 5), what changed this pass (Section 10 + the frontend callout below), and an end-to-end test flow (Section 9)

1. What you're testing

A FastAPI backend covering Phase 1 (Authentication, Candidate Portal, Resume Upload, Resume Parser), Phase 2 (Candidate Profile Engine, Benchmark Repository, Matching Engine), and now **Phase 3 (Career Intelligence — Gap Analysis, Recommendations, Career Roadmap)** of the platform. Nine modules are implemented and testable end-to-end today:

- **Module 1 — Authentication & Access Management** (JWT access/refresh, RBAC, password reset, admin user management)
- **Module 2 — Candidate Management System** (profile, skills, education, experience, resume upload/versioning)
- **Module 3 — Resume Intelligence Engine** (text extraction + rules-based structured parsing)
- **Module 4 — Candidate Profile Database** (paginated, filterable search for company staff/admins)
- **Module 5 — Benchmark Repository** (managed by company staff and admins)
- **Module 6 — Benchmark Matching Engine** (Match Score, Readiness Score, Gap Summary)
- **Module 7 — Gap Analysis** (detailed gap report: skill gaps, missing certifications, experience gap, resume-quality issues)
- **Module 8 — Recommendations** (learning items, certification suggestions, resume improvements, career-development advice)
- **Module 9 — Career Roadmap** (phased 30/90/180-day + 12-month action plan)

Since the last QA pass: Phase 3 (Career Intelligence) has been integrated and is now testable end-to-end via the new `/career` endpoints (Section 5.7) — it was a standalone, not-yet-wired package at the last pass. Generation is asynchronous (background task, like resume parsing) and produces three sections on one plan row per candidate: a gap report, recommendations, and a roadmap. It runs a local, deterministic pipeline by default and *optionally* enriches the output with a local LLM (Ollama) that **degrades gracefully** to the deterministic result if the model is unreachable — see Sections 2, 7 and 10. Two working-tree fixes landed this pass to make it run on this environment (Section 10).

Phases 4-6 (Recruiter & HR portals as distinct capability sets, Employer portal, Consultation & Learning marketplace) are **not built yet** — do not file bugs against endpoints that don't exist yet; check the endpoint list in Section 5 first.

For the frontend developer — new/changed contracts this pass (build/update the UI for these):

- **Password reset is now a 6-digit code, not a link.** Rework the reset page: collect *email* + *6-digit code* + *new password* and POST `/auth/password-reset/confirm {email, code, new_password}` (was `{token, new_password}`; stop reading `?token=` from the URL). Request stays POST `/auth/password-`

reset/request {email}. Show a generic "invalid or expired code" on failure and a "resend code" action. (Section 5.1)

- **Scorecard (new, no UI yet).** GET /scorecard/me returns the employability metrics + per-skill scores — build a scorecard page/section to render the Metric/Current/Target/Status cards and the per-skill table. Let candidates set a skill's 1-5 self-rating via PATCH /candidates/me/skills/{id} {manual_score}. (Section 5.9)
- **Recommendations (Phase 3) still has no UI.** GET /career/me already returns a recommendations object (learning, certifications, resume improvements, career development) that no page renders yet — add a Recommendations view. (Gap Analysis and Career Roadmap pages already exist.)

2. Tech stack (so error messages make sense)

- **FastAPI + SQLAlchemy 2.0 + Alembic** — the API framework and DB layer
- **PostgreSQL + pgvector** — relational data, plus vector similarity for matching
- **JWT (access + refresh tokens)** — stateless auth, no server-side sessions
- **Local, free parsing/embeddings** — OpenAI is intentionally disabled for resume parsing (no API key configured). Resume parsing uses a regex/keyword rules engine only; embeddings use scikit-learn's HashingVectorizer, not an LLM. This is expected, not a bug — see Section 7.
- **Phase 3 LLM enrichment (optional, local)** — career-plan narratives are optionally generated by a local **Ollama** model through its OpenAI-compatible endpoint (default provider ollama, base URL `http://localhost:11434/v1`). This is *separate* from resume parsing and from embeddings. If Ollama is down, the model is missing, or `PHASE3_LLM_ENABLED=false`, the pipeline falls back to fully deterministic output and never fails the request. The response's `llm_used` flag tells you which path ran.
- **Resume storage** — files are stored as blobs directly in Postgres, not S3 (also intentional for now).

3. Getting the environment running

3.1 Start Postgres (Docker Desktop must be running first):

```
docker compose up -d db
```

Maps to host port **5433**, not 5432 — 5432 is commonly already in use locally.

3.2 Python environment:

```
python -m venv .venv
.venv\Scripts\pip install -r requirements.txt
```

3.3 Copy env file and run migrations:

```
cp .env.example .env
.venv\Scripts\python -m alembic upgrade head
```

Migrations now run through `0005_career_plans` (the Phase 3 `career_plans` table). If `alembic upgrade head` stops before that revision, Phase 3 endpoints will 500 on first use.

3.4 Run the server:

```
.venv\Scripts\python -m uvicorn app.main:app --reload
```

Interactive docs at <http://127.0.0.1:8000/docs> (Swagger UI).

3.5 (Optional) Phase 3 LLM enrichment via Ollama. Career plans work without this — you'll just get deterministic narratives (`llm_used=false`). To exercise the LLM path, run a local Ollama with a model pulled, and point the backend at it via env vars (all optional, with defaults):

```
PHASE3_LLM_ENABLED=true           # default true
PHASE3_LLM_PROVIDER=ollama        # default ollama
PHASE3_LLM_BASE_URL=http://localhost:11434/v1
PHASE3_LLM_MODEL=qwen2.5:3b       # default is llama3.1:8b – see note
PHASE3_LLM_FALLBACK_MODEL=qwen2.5:7b # stronger model, tried only if the primary's output
won't conform; blank to disable
PHASE3_LLM_TIMEOUT=60            # seconds, default 90
```

Model choice matters for latency. The default `llama3.1:8b` did *not* finish within a reasonable time on the current test machine (generation ran past 7 minutes and the plan stayed at processing). Switching to a smaller model — `qwen2.5:3b` — completes a full plan in roughly **40–110 seconds**. If your career plans appear to hang at processing, suspect the model/hardware combo before the code. Set `PHASE3_LLM_MODEL` to a small model, or set `PHASE3_LLM_ENABLED=false` to skip the LLM entirely.

Port-in-use error? "WinError 10013" or "address already in use" on port 8000 almost always means another uvicorn instance (yours or someone else's test run) is still bound to that port. Check for a stray python process before assuming the app itself is broken.

4. Roles and how to get a test account for each

There are **6 roles**: **candidate**, **recruiter**, **hr_reviewer**, **employer**, **administrator**, and **super_admin**. (An intermediate build briefly collapsed recruiter/hr_reviewer/employer into a single "employee" role — that was reverted; accounts from that window were remapped to recruiter as a default, and can be individually reassigned — see "Reassigning a user's role" below.)

Role summary

- **candidate** — self-registers via `POST /auth/register`. Owns and edits their own profile, skills, education, experience, and resumes. Can read (not manage) benchmarks, compute/view only their own matches, and **generate/view only their own career plan** (gap report, recommendations, roadmap).
- **recruiter / hr_reviewer / employer** — company staff, referred to collectively as "company staff" below since they currently share identical permissions. Cannot self-register — created only by an administrator (Section 5.2). Each can view every candidate's profile, search/filter candidates, compute/view matches for any candidate, **generate/view any candidate's career plan**, and create/edit/delete benchmarks. None can edit a candidate's own profile data. The three labels exist for organizational/reporting clarity — permissions do not currently differ between them.
- **administrator** — everything company staff can do, plus user management for *non-admin* accounts: create/list/activate/deactivate candidates and staff, and reassign roles among candidate/recruiter/hr_reviewer/employer. As of this pass, an administrator can **no longer create**,

promote to, or modify administrator/super_admin accounts — that is now exclusively the super_admin's job (see below).

- **super_admin** — a narrowly-scoped role whose **only** job is CRUD over admin-tier accounts (administrator and super_admin): create them, list them, switch between administrator/super_admin, and deactivate/reactivate them, via /super-admin/admins (Section 5.8). A super_admin has **no** access to candidate/staff management, benchmarks, matching, or any candidate data — the /admin, /candidates, /career/me, etc. routes return 403 for them. Guard rails: a super_admin cannot demote/deactivate their own account, and the last active super_admin cannot be demoted/deactivated (lockout prevention).

Getting a candidate account

Just register normally: POST /auth/register with "role": "candidate" (or omit role — it's the default). This is the only role the public endpoint will accept for a brand-new email.

Getting an administrator account

Admins are designated by an email allowlist, not a role field. The server has an ADMIN_EMAILS env var (comma-separated). If you register with an email on that list, you are **automatically granted the administrator role** regardless of what role you passed — every other email is rejected if it tries to register as anything other than candidate. Ask whoever manages the environment which email(s) are on the allowlist, then simply POST /auth/register with that email like any candidate signup.

If you need a DB-level admin outside the normal register flow (e.g. bootstrapping a fresh environment with no allowlisted email yet), the provisioning script still works:

```
python provision_admin.py <email> <password> ["Full Name"]
```

Getting a super_admin account

Super admins are designated by their own allowlist — the SUPER_ADMIN_EMAILS env var (comma-separated), which takes **precedence** over ADMIN_EMAILS if an email is on both. Register normally with an allowlisted email and you are granted super_admin. Bootstrap without an allowlist using the provisioning script:

```
python provision_super_admin.py <email> <password> ["Full Name"]
```

A super_admin then manages all administrator/super_admin accounts via /super-admin/admins (Section 5.8).

Getting a recruiter / hr_reviewer / employer account

Company staff cannot self-register under any circumstances — POST /auth/register rejects any role except candidate with a 422, and there is no public staff-signup path. An administrator must create the account:

```
POST /api/v1/admin/users
Authorization: Bearer <admin access token>
```

```
{"email": "staff@company.com", "password": "...", "full_name": "...", "role": "recruiter"}
```

role defaults to recruiter if omitted and accepts candidate/recruiter/hr_reviewer/employer. It **no longer accepts administrator or super_admin** — those return 403 (admin-tier accounts are created only by a super_admin, Section 5.8). Accounts created this way are marked verified immediately (no email confirmation step exists in this backend).

Reassigning a user's role

An administrator can change any existing user's role after the fact:

```
PATCH /api/v1/admin/users/{user_id}/role
Authorization: Bearer <admin access token>
```

```
{"role": "hr_reviewer"}
```

An admin cannot change their own role this way (400 if attempted — prevents accidentally locking yourself out of admin access). Reassigning a user **into** candidate automatically creates a candidate profile for them if they don't already have one; reassigning a candidate **out of** that role leaves their existing profile data in place rather than deleting it.

Security note: POST /api/v1/auth/register will reject "role": "recruiter", "hr_reviewer", "employer", or "administrator" with a 422 (unless the email is on the admin allowlist, in which case it's silently upgraded to administrator instead of rejected). This is intentional — do not report either behavior as a bug.

5. Full endpoint reference

All routes are prefixed with /api/v1 unless noted. "Auth required" lists which role(s) may call it; "any" means any authenticated role.

5.1 Authentication — /auth

Method	Path	Auth required	Notes
POST	/auth/register	none (public)	JSON body. role defaults to candidate; any other role is rejected (422) unless the email is on the admin allowlist (auto-granted administrator instead).
POST	/auth/login	none (public)	form-urlencoded (username, password), NOT JSON. Response body is {access_token, token_type}; the refresh token is set only as an httpOnly cookie (XSS hardening), never in the JSON body.
POST	/auth/refresh	none (needs refresh cookie)	Reads the refresh token from the httpOnly cookie only (a JSON {refresh_token} body is ignored). Rotates the token (old one revoked — reuse gives 401) and returns a fresh access token. Send with credentials/cookies enabled.
POST	/auth/logout	none (needs refresh cookie)	Revokes the refresh token carried in the httpOnly cookie and clears it.
GET	/auth/me	any authenticated	Returns current user.
POST	/auth/password-reset/request	none (public)	JSON {email}. Emails a 6-digit code (15-min expiry). Always 202, even for unknown emails (no enumeration). Invalidates any prior unused code for that user.
POST	/auth/password-reset/confirm	none (needs code)	JSON { email, code, new_password } (code = the 6 digits emailed). 200 on success. One generic 400 "Invalid or expired code" for every failure (wrong/expired/unknown/locked). Code locks after 5 wrong attempts.

5.2 Administration — /admin (administrator only)

Method	Path	Auth required	Notes
POST	/admin/users	administrator	JSON {email, password, full_name, role}. role defaults to recruiter; accepts candidate/recruiter/hr_reviewer/employer. Passing administrator or super_admin returns 403 (admin-tier is super_admin-only). This is the way to create staff accounts.
GET	/admin/users	administrator	Query params: role?, limit, offset. Lists all users, optionally filtered by role.
PATCH	/admin/users/{user_id}/role	administrator	JSON {role}. Reassigns a non-admin user's role. 400 if targeting your own account; 403 if the target is admin-tier or the requested role is administrator/super_admin. Reassigning into candidate auto-creates a profile if missing.
PATCH	/admin/users/{user_id}/deactivate	administrator	Sets is_active=false. 400 on your own account; 403 if the target is admin-tier.
PATCH	/admin/users/{user_id}/activate	administrator	Re-enables a deactivated user. 403 if the target is admin-tier.

5.3 Candidate Management — /candidates

Method	Path	Auth required	Notes
GET	/candidates/me	candidate	Own profile incl. skills/education/experience.
PUT	/candidates/me	candidate	Update profile fields — all optional, partial updates supported.
POST	/candidates/me/skills	candidate	JSON {name, proficiency?, category?, manual_score?}. manual_score is the user's 1-5 self-rating used by the scorecard (422 if outside 1-5).
PATCH	/candidates/me/skills/{skill_id}	candidate	Partial update of an existing skill — primarily to set manual_score (1-5); also proficiency/category. 404 if not the caller's skill.
DELETE	/candidates/me/skills/{skill_id}	candidate	
POST	/candidates/me/education	candidate	JSON {degree?, field_of_study?, institution?, start_date?, end_date?, grade?}.
DELETE	/candidates/me/education/{education_id}	candidate	
POST	/candidates/me/experience	candidate	JSON {title?, company?, industry?, start_date?, end_date?, is_current?, description?}.
DELETE	/candidates/me/experience/{experience_id}	candidate	
POST	/candidates/me/experience/recalculate	candidate	Clears the manual total-experience override and re-derives it from experience entries.
GET	/candidates	recruiter, hr_reviewer, employer, administrator	Query params: industry, functional_area, experience_level, min/max_experience_years, skill, limit, offset. Candidate role gets 403. Returns a paginated envelope {items, total, limit, offset}, not a bare list.
GET	/candidates/{candidate_id}	recruiter, hr_reviewer, employer, administrator	404 if candidate_id doesn't exist. Response includes full_name/email from the account.

5.4 Resume Intelligence — /resumes

Method	Path	Auth required	Notes
POST	/resumes/upload	candidate	multipart/form-data, field name 'file'. pdf/doc/docx only (400 otherwise), 10MB max (400 otherwise). Parsing runs in background — status starts 'pending'.
GET	/resumes	candidate	Lists all versions for the current candidate.
GET	/resumes/{resume_id}	candidate	Returns parsing_status, parsing_method, parsed_data.
GET	/resumes/{resume_id}/download	candidate	Streams the raw file back (not a presigned URL — storage is Postgres, not S3).
POST	/resumes/{resume_id}/reparse	candidate	Re-runs parsing on an existing upload.
DELETE	/resumes/{resume_id}	candidate	

After a successful upload, poll GET /resumes/{id} until parsing_status is completed (usually 1-3 seconds) before asserting on parsed_data or on the candidate profile having been auto-populated.

5.5 Benchmark Repository — /benchmarks

Method	Path	Auth required	Notes
POST	/benchmarks	recruiter, hr_reviewer, employer, administrator	JSON — name, category, level required; required_skills/preferred_skills/certifications etc. optional lists.
GET	/benchmarks	any authenticated	Query params: category, level, functional_area, is_active. Candidates can read but not manage benchmarks.
GET	/benchmarks/{benchmark_id}	any authenticated	
PUT	/benchmarks/{benchmark_id}	recruiter, hr_reviewer, employer, administrator	All fields optional (partial update).
DELETE	/benchmarks/{benchmark_id}	recruiter, hr_reviewer, employer, administrator	Soft-delete — sets is_active=false, does not remove the row.

5.6 Matching Engine — /matching

Method	Path	Auth required	Notes
POST	/matching/me/compute	candidate	Optional ?benchmark_id=... to target one; otherwise computes against all active benchmarks (filtered by the candidate's industry if set — see Section 8).
GET	/matching/me	candidate	Lists previously computed matches.
POST	/matching/candidates/{candidate_id}/compute	recruiter, hr_reviewer, employer, administrator	
GET	/matching/candidates/{candidate_id}	recruiter, hr_reviewer, employer, administrator	404 if candidate_id doesn't exist.

Each match result includes `match_score`, `readiness_score`, `semantic_similarity`, `matched/missing skill` and `certification lists`, `experience_gap_years`, and a structured `gap_summary` object.

5.7 Career Intelligence (Phase 3) — /career

Generation is **asynchronous** (background task, like resume parsing). POST `.../generate` returns **202** with the plan row at `status=pending`; poll the GET endpoint until `status` is `completed` (or `failed`). The section endpoints (`gap-report` / `recommendations` / `roadmap`) return **409** until the plan is `completed`. If the candidate has no computed benchmark matches yet, generation computes them first so the plan isn't built from an empty gap set.

Method	Path	Auth required	Notes
POST	/career/me/generate	candidate	202 Accepted. Starts (or re-runs) the current candidate's plan; returns status row. 409 if a generation is already in progress (status=processing). 404 if the caller has no candidate profile.
GET	/career/me	candidate	Full plan: status, error, llm_used, generated_at, and the gap_report / recommendations / roadmap objects (null until completed). 404 if no plan generated yet.
GET	/career/me/gap-report	candidate	Just the gap report. 409 if the plan isn't completed yet.
GET	/career/me/recommendations	candidate	Just the recommendations. 409 if not completed.
GET	/career/me/roadmap	candidate	Just the roadmap. 409 if not completed.
POST	/career/candidates/{candidate_id}/generate	recruiter, hr_reviewer, employer, administrator	202 Accepted. Generate/regenerate any candidate's plan. 404 if candidate_id doesn't exist.
GET	/career/candidates/{candidate_id}	recruiter, hr_reviewer, employer, administrator	Full plan for any candidate. 404 if candidate_id doesn't exist.

The `gap_report` includes `overall_readiness_score`, `overall_recommendation`, target benchmarks, skill gaps (with priority), missing certifications, an experience gap, resume-quality issues, and a narrative. `recommendations` includes learning items, certification suggestions (with provider/URL), resume improvements, and career-development advice. `roadmap` includes phased 30-day / 90-day / 180-day / 12-month goals and actions, plus a `generated_by` marker (deterministic or `llm:...`). The `llm_used` flag on the plan row is the quick way to see whether the LLM path or the deterministic fallback produced this plan.

5.8 Super Administration — /super-admin (super_admin only)

Exclusive CRUD over admin-tier accounts (administrator + super_admin). Every route here is super_admin-only; administrators, staff, and candidates get 403. Non-admin targets return 404 (so the surface never confirms whether a given id is a candidate/staff account).

Method	Path	Auth required	Notes
POST	/super-admin/admins	super_admin	JSON {email, password, full_name, role}. role must be administrator or super_admin (422 otherwise). The only way to create admin-tier accounts. 409 on duplicate email.
GET	/super-admin/admins	super_admin	Lists admin-tier accounts. Query params: role? (administrator super_admin — 400 for any other), limit, offset.
PATCH	/super-admin/admins/{user_id}/role	super_admin	JSON {role} constrained to administrator super_admin (422 otherwise). 400 on your own account or if it would demote the last active super_admin; 404 if the target isn't admin-tier.
PATCH	/super-admin/admins/{user_id}/deactivate	super_admin	Soft-delete (is_active=false). 400 on your own account or the last active super_admin; 404 if not admin-tier.
PATCH	/super-admin/admins/{user_id}/activate	super_admin	Reactivates an admin-tier account. 404 if not admin-tier.

5.9 Scorecard — /scorecard

An employability scorecard computed on demand from current profile data (no stored row — every call recomputes). Consolidates existing signals (benchmark readiness, resume-quality issues, project/certification counts) with a new per-skill score.

Method	Path	Auth required	Notes
GET	/scorecard/me	candidate	Own scorecard. 404 if no candidate profile.
GET	/scorecard/candidates/{candidate_id}	recruiter, hr_reviewer, employer, administrator	Any candidate's scorecard. 404 if not found. Candidate role gets 403.

Response shape: employability_score, technical_skills_score, resume_score, projects_count, certifications_count, benchmark_readiness (best readiness across computed matches, or null), a metrics array (each {metric, current, target, unit, status} where status ∈ Good/Medium/Improve/Critical), and a skill_scores array (each {name, proficiency, manual_score, in_resume, in_project, score 0-100}).

How a skill's score is computed: it blends *evidence the skill is real* — parsed from the resume (skill source="resume" → in_resume) and its name appearing in a project (in_project) — with proficiency (beginner/intermediate/advanced/expert) and the candidate's manual 1-5 self-rating (set via POST/PATCH /candidates/me/skills). Manually-added skills with no resume/project evidence score lower; add evidence or a manual score to raise them. Technical Skills = average of per-skill scores; Employability blends technical, resume, projects and certifications.

6. Testing via Swagger UI (/docs) — things that trip people up

- **Login is a form, not JSON.** POST /auth/login renders as a username/password form in Swagger — don't paste a JSON body into it.
- **Authorize button expects the raw token, no "Bearer" prefix.** Click the padlock icon top-right, paste only the JWT string. Swagger adds "Bearer " automatically. Pasting "Bearer eyJ..." yourself produces "Bearer Bearer eyJ..." and every request 401s.
- **Switching users requires re-authorizing.** Click Authorize → Logout → log in as the other user → Authorize again with the new token. There's no per-request override in the Swagger UI.
- **Resume upload is a file picker,** not a JSON/text box — click "Choose File" under Try it out.
- **Career plans are async — don't read the section endpoints immediately.** After POST /career/me/generate, the gap-report/recommendations/roadmap endpoints return 409 until generation finishes. Poll GET /career/me and wait for status=completed first. With the LLM path enabled this can take tens of seconds to a couple of minutes (Section 3.5).
- **A 401 right after a successful login** almost always means a copy-paste error in the token (extra "Bearer", truncated string, or using the refresh_token where an access_token is expected).

7. Known and expected behavior (not bugs)

- **parsing_method is always "rules", never "llm" or "hybrid".** The OpenAI-based LLM fallback for low-confidence resume parses is intentionally disabled (no OpenAI account). Rules-only parsing runs for every resume. (This is unrelated to the Phase 3 career-plan LLM, which is a separate local Ollama path.)
- **Phase 3 career plans degrade gracefully when the LLM is unavailable.** If Ollama is down/missing or PHASE3_LLM_ENABLED=false, generation still completes with fully deterministic narratives and llm_used=false; the plan's notes array explains why the LLM was skipped. A completed plan with llm_used=false is expected in that configuration, not a failure.
- **Career-plan generation latency depends entirely on the LLM model.** Deterministic-only plans finish in about a second; LLM-enriched plans take tens of seconds to minutes depending on the model and hardware (Section 3.5). A plan sitting at processing for a while is usually the model working, not a hang — but see Section 8 for the genuine stuck-plan gap.
- **Phase 3 structured LLM output is schema-constrained.** The narrative/roadmap calls use grammar-constrained decoding (Ollama json_schema response_format), so even the small qwen2.5:3b model reliably emits conforming JSON — all four roadmap horizons, non-empty narratives. If the primary model's output still fails to conform, the provider retries once with PHASE3_LLM_FALLBACK_MODEL (a stronger local model) before falling back to deterministic output. A deterministic-marked section (generated_by: deterministic) therefore now means the LLM genuinely failed twice or is disabled — not just small-model flakiness.
- **semantic_similarity uses local word/bigram hashing, not an LLM embedding.** It rewards literal keyword overlap but will not catch synonyms (e.g. "ML" vs "machine learning"). Do not expect GPT-level semantic understanding here.
- **Skills/phone extraction is keyword/pattern-based, not NLP.** The skills dictionary and phone regex were broadened in an earlier fix pass, but they remain fixed dictionaries/patterns. A skill or number format outside what's listed will be missed; that's a coverage gap to note, not grounds to fail a ticket on its own.

- **No rate limiting is implemented.** Rapid repeated requests will not get 429s. Known gap for pre-production.
- **Resume files live in Postgres, not S3.** Download uses a direct byte-streaming endpoint, not a presigned URL.
- **is_verified exists on the user model but nothing enforces it at login.** An unverified user can still log in. Flag it if you think it should block login, but it may be intentional.
- **The first matching/compute call for a candidate is slower than subsequent ones.** Building the candidate's embedding happens lazily on first use — a one-time cold-start latency cliff, then faster warm calls. The same applies transitively to the first career-plan generation, which computes matches if none exist.

8. Known gaps — do not re-report these, they're already tracked

- **Career plans can get stuck at status=processing if the server restarts mid-generation.** The background task sets the row to processing; if the process dies before it finishes, nothing resets it, and `POST .../generate` then returns 409 ("already in progress") permanently — the candidate can never regenerate. Recovery currently requires manually resetting the row in the DB. (Flagged for a startup/staleness fix.)
- **Industry/category mismatch silently yields an empty career plan.** Matching filters active benchmarks by `benchmark.category ILIKE %candidate.industry%` when the candidate has an industry set. If the candidate's industry string doesn't substring-match any benchmark category (e.g. industry "Software" vs category "Information Technology"), matching returns 0 results and the career plan completes but with an empty gap set and hollow narrative — no warning is surfaced. Set the candidate's industry to match your benchmark categories (or leave it blank) when setting up a meaningful plan.
- **No dedicated frontend pages for gap analysis / roadmap yet.** The `/career` endpoints are backend-only for now; the candidate-facing UI pages are not built.
- **Password reset token is never emailed** — it must be read directly from the database to complete the flow end-to-end.
- **No MIME-sniffing or malware scanning on resume upload** — only the file extension is checked.
- **No OCR** — scanned/image-only PDFs (no real text layer) will fail to parse meaningfully.
- **Single-instance/single-worker only** — the connection pool was sized for higher concurrency, but multi-process/horizontal scaling has not been tested.
- **Resume upload holds a DB connection while writing the file blob** and updating prior-version status, adding latency under load (mitigated by the pool resize, but the handler hasn't been restructured to release the connection earlier).
- **Phases 4-6** (distinct recruiter/HR/employer capability sets, consultation & learning marketplace) do not exist as backend endpoints yet.

9. Suggested end-to-end test flow

- 1. Register a candidate via `/auth/register`. Confirm registering with `role=recruiter/hr_reviewer/employer/administrator` is rejected (422), UNLESS you're using an allowlisted admin email (then it should succeed as `role=administrator`).
- 2. Log in as the candidate. Confirm `/auth/me` returns the right user, and an unauthenticated request 401s.

- 3. Log in as an admin (allowlisted email, or provision_admin.py). Use POST /admin/users to create a recruiter account. Confirm a candidate or the new recruiter both get 403 on /admin/users.
- 4. As the admin, PATCH /admin/users/{recruiter_id}/role to hr_reviewer; confirm the account still logs in with company-staff permissions. Confirm the admin gets 400 changing their own role.
- 5. Build out the candidate profile: PUT /candidates/me (set industry to match your benchmark categories — see Section 8), add 3-5 skills, one education entry, one experience entry.
- 6. Upload a real .pdf or .docx resume. Poll GET /resumes/{id} until parsing_status is completed.
- 7. Re-check GET /candidates/me — confirm skills/certifications/education/experience from the resume merged in and profile_completeness increased.
- 8. Confirm the company-staff account CAN search/view that candidate (GET /candidates, GET /candidates/{id}) and the candidate role CANNOT (403). Confirm GET /candidates returns the paginated {items, total, limit, offset} shape with full_name/email.
- 9. As company staff (or admin), create 1-2 benchmarks whose category matches the candidate's industry, with skills relevant to the candidate, via POST /benchmarks. Confirm the candidate gets 403 on POST but CAN GET /benchmarks.
- 10. As the candidate, POST /matching/me/compute and inspect match_score, readiness_score, gap_summary. Confirm you get a non-zero number of matches (0 matches usually means the industry/category mismatch in Section 8).
- 11. As company staff, compute/view matches for that candidate via /matching/candidates/{id} and confirm they agree with what the candidate saw. Confirm GET /matching/candidates/{unknown-id} returns 404, not 200 [].
- 12. **Phase 3:** As the candidate, POST /career/me/generate (202). Poll GET /career/me until status=completed. Then GET /career/me/gap-report, /recommendations, and /roadmap and confirm each returns populated content consistent with the skills/gaps set up. Note whether 11m_used is true or false and whether the roadmap has all four horizons.
- 13. **Phase 3:** As company staff/admin, GET /career/candidates/{id} for that candidate and confirm it returns the same plan; POST /career/candidates/{id}/generate to regenerate and confirm it completes. Confirm a candidate token is rejected (403) on the /career/candidates/* routes and a staff token is rejected (403) on /career/me.
- 14. **Scorecard:** As the candidate, PATCH a skill's manual_score (1-5), then GET /scorecard/me — confirm the metrics (Employability/Technical/Resume/Projects/Certifications with Current/Target/Status) and per-skill scores return, and that the rated skill's score moved. As staff, GET /scorecard/candidates/{id}; confirm a candidate token is 403 there.
- 15. **Password reset (OTP):** POST /auth/password-reset/request {email} → 202; read the 6-digit code from the email (or DB in dev); POST /auth/password-reset/confirm {email, code, new_password} → 200; confirm old password now fails and new works; confirm a wrong code returns the generic 400 and locks after 5 tries.
- 16. Negative/edge cases: wrong password, expired/reused refresh token, oversized/unsupported file upload, unknown candidate/benchmark IDs, cross-role access attempts, and calling the career section endpoints before generation completes (expect 409) or before any plan exists (expect 404).
- 15. If load-testing, fire 50-100 concurrent requests at a DB-backed endpoint (e.g. POST /matching/me/compute) and confirm 200s (or at worst a graceful 503 with Retry-After), not raw 500s.

10. Changelog since the last QA pass

New: Employability Scorecard (Section 5.9)

- **New /scorecard endpoints** — GET /scorecard/me (candidate) and GET /scorecard/candidates/{id} (staff/admin) return an on-demand employability scorecard (Metric/Current/Target/Status) built from a new per-skill score plus existing readiness/resume/gap/cert/project signals.
- **Per-skill manual score** — CandidateSkill gained manual_score (1-5 self-rating). POST /candidates/me/skills accepts it and a new PATCH /candidates/me/skills/{id} sets/updates it (migration 0008). The model score blends this with resume/project evidence and proficiency.
- **Verified:** scorecard returns 200 with all metrics + per-skill scores; setting manual_score=5 raised a skill's score (52→66); invalid score (9) → 422; RBAC holds (candidate 200 on /me, 403 on /candidates/{id}, 401 unauth).
- **Not built (needs a candidate population or separate subsystems):** peer comparison, ranking & percentile, project/progress trackers, interview/communication/leadership scores.

Changed: password reset is now a 6-digit code (OTP)

- Reset switched from a link-token to an emailed **6-digit code**. POST /auth/password-reset/request {email} emails the code (15-min expiry); POST /auth/password-reset/confirm now takes {email, code, new_password} (was {token, new_password}). Code locks after 5 wrong attempts; all failures return one generic 400. Migration 0007 (drops the unique constraint on the token column, adds an attempts counter). Verified end-to-end incl. real SMTP delivery.

New: super_admin role & admin-tier lockdown

- **New super_admin role** (Section 4) whose sole job is CRUD over administrator/super_admin accounts, via the new /super-admin/admins endpoints (Section 5.8). Granted by the SUPER_ADMIN_EMAILS allowlist (precedence over ADMIN_EMAILS) or the provision_super_admin.py bootstrap script. Guard rails: no self-demote/deactivate, and the last active super_admin is protected from demotion/deactivation. No schema migration was needed (role is a free string column).
- **Admin-tier lockdown.** Regular administrators can no longer create, promote to, or modify administrator/super_admin accounts through /admin — those now return 403 (Section 5.2). Admin-tier lifecycle is exclusively the super_admin's. Admins keep full candidate/staff management.
- **Verified:** 22/22 backend checks (create/list/role-change/deactivate/activate, non-admin-tier role rejected 422, self-protection 400, last-super-admin guard, 404 on non-admin targets, RBAC 403 for admin/candidate, admin lockdown) plus a browser walkthrough of the new /super-admin page.

QA findings fixed this pass (from the Phase 3 QA report)

- **F-3 (concurrency guard) — fixed.** POST /career/*/generate could enqueue a duplicate background generation: a second request during the brief pending window returned 202 instead of 409, because the guard only checked processing while _start_generation set pending first. Fixed in career.py: the guard now treats pending as in-progress, locks the plan row with SELECT ... FOR UPDATE for an atomic check-and-set, and catches the candidate_id unique-constraint race on first-time creation. Re-tested: second generate during pending/processing → 409; a 6-way concurrent burst → all 409; regeneration after completed → 202.

- **F-2 (.env.example gitignored) — fixed.** The broad `.env*` ignore rule also matched the committed template, so a fresh clone had nothing for `cp .env.example .env` to copy. Added `!.env.example / !backend/.env.example` exceptions and committed the template (now including the ADMIN/SUPER_ADMIN/PHASE3 settings).
- **F-1 (/auth/refresh contract) — documented.** Not a code defect: the refresh token travels only as an `httpOnly` cookie by design (XSS hardening), so the endpoint ignores a JSON `{refresh_token}` body. Section 5.1 has been corrected to describe the cookie-based contract.

The headline change of the prior pass was **Phase 3 (Career Intelligence) going live** — now wired into the API and testable end-to-end.

Phase 3 integration — Modules 7, 8, 9

- **New /career endpoints** (Section 5.7): candidate self-service (`/career/me*`) and company-staff/admin access to any candidate (`/career/candidates/{id}*`). Generation is an async background task; the plan row moves `pending` → `processing` → `completed/failed`, exactly like resume parsing.
- **New migration 0005_career_plans** adds the `career_plans` table (one row per candidate, regenerated in place). Run `alembic upgrade head`.
- **Auto-computes matches first.** If a candidate has no benchmark matches when a plan is requested, generation runs matching before building the gap set, so a first-time plan isn't empty for lack of matches.
- **Verified this pass:** candidate flow (`generate` → `poll` → `gap-report/recommendations/roadmap`) and staff flow (`view` + `regenerate` any candidate) both return populated output; RBAC holds both directions (candidate 403 on `/career/candidates/*`, staff 403 on `/career/me`); the LLM-enriched path produced `llm_used=true` plans with real narratives via Ollama `qwen2.5:3b`.

Working-tree fixes applied this pass

- **Phase 3 config now loads .env.** `phase3/config.py` reads `PHASE3_*` vars via `os.getenv`, but nothing loaded the `.env` file into the environment (the app's `pydantic-settings` reads it privately), so every `PHASE3_*` setting in `.env` was silently ignored. Added an explicit `load_dotenv()` call so these settings take effect.
- **Default LLM model switched for the test machine.** `PHASE3_LLM_MODEL=qwen2.5:3b` and `PHASE3_LLM_TIMEOUT=60` were set in `backend/.env` because the code default `llama3.1:8b` ran past 7 minutes without completing on this hardware; the smaller model finishes in ~40-110s. (These are env changes, not code — adjust per environment.)

Phase 3 recommendation & LLM-reliability fixes (this pass)

- **Duplicate recommendation fixed.** A certification could appear twice — once in `learning` (pulled from a skill's catalog entry, e.g. CKA under 'kubernetes') and again in the dedicated `certifications` list (from `missing_certifications`). The recommender (`phase3/recommendations.py`) now dedupes both lists against one shared identity set (`title+provider`) and routes certification-type resources to the `certifications` bucket, so nothing is recommended twice and certs never appear under 'learning'.
- **LLM structured output made reliable via constrained decoding.** The recommendation-narrative and roadmap LLM calls were frequently failing on `qwen2.5:3b` — the model echoed the JSON schema back or returned empty phases, forcing a deterministic fallback (observed ~2 of 3 roadmap runs). Root cause: the provider used `json_object` mode, which only guarantees valid JSON, not schema conformance. Fixed in `phase3/llm/openai_compatible.py`: switched to grammar-constrained decoding (`json_schema_response_format`), added a conformance check, and a one-shot retry on a stronger fallback model.

(PHASE3_LLM_FALLBACK_MODEL, default qwen2.5:7b) before deterministic fallback. Re-tested: the 3B model now produces all four roadmap horizons and non-empty narratives reliably, with no fallback needed.

New / updated gaps found this pass

- **Stuck-plan-on-restart** (Section 8): a server restart mid-generation leaves the plan at processing and blocks regeneration with a permanent 409. No automatic recovery yet.
- **Silent empty plan on industry/category mismatch** (Section 8): matching's category `ILIKE %industry%` filter can yield 0 matches and a hollow plan with no warning.
- **Sparse course catalog**: some in-demand skills (e.g. terraform, go) have no catalog entry, so they surface in the gap report and roadmap but get no learning resource; a notes entry flags this per plan.

Carried forward from the previous QA pass (still current)

- **BK-1** — DB connection-pool exhaustion under load fixed (pool resized to 20 + 40 overflow, configurable; SQLAlchemy TimeoutError/OperationalError now mapped to a graceful 503 + Retry-After).
- **BK-2** — GET `/matching/candidates/{id}` now returns 404 (not 200 []) for a non-existent candidate.
- **BK-3** — resume phone extraction now handles international formats (E.164 digit-count validation; date ranges still correctly rejected).
- **O-1** — skills dictionary expanded (~30 → ~115 entries) with fixed word-boundary matching.
- **O-3** — `provision_admin.py` present at repo root; `.env.example` present.
- **Role model** — 5 distinct roles restored (migration `0003_uncollapse_roles`); admin role-reassignment endpoint; recruiter/hr_reviewer/employer still share identical permissions.
- **Candidate search** — paginated `{items, total, limit, offset}` envelope with `full_name/email` enrichment; total computed against the un-joined query to avoid row fan-out.

Generated for internal QA handoff — updated 2026-07-20 for Phase 3 (Career Intelligence). If anything here is out of date with the running API, the code and OpenAPI schema at `/docs` are the source of truth — flag the discrepancy so this guide can be updated.